# Azure Interview Guide — 20 Questions from Champion Portal Deployment

**Project:** Champion Portal (.NET 9 API + React SPA + Azure SQL)
**Purpose:** Detailed Q&A with context to drive technical interviews
**Based on:** Hands-on Azure deployment completed in practice

---

## Question 1

**Interviewer:** Walk me through how you deployed your application to Azure. What steps did you follow from start to finish?

**Context to set:** This is your flagship answer. You did a full manual deployment without GitHub CI/CD. Show you understand the full pipeline, not just "I clicked deploy."

**Answer:**

I deployed Champion Portal—a .NET 9 Web API with a React frontend and SQL Server database—to Azure using a manual ZIP deployment approach.

**Step 1 — Provision Azure resources:**

- Created a **Resource Group** ( `rg-championportal` ) in Central India to group all related resources.
- Created **Azure SQL Database** ( `ChampionPortal` ) on a new SQL Server ( `championportal-sql-ashish` ) using Basic tier for cost-effective practice.
- Created an **App Service** ( `championportal-AI` ) on Windows with .NET 9 runtime and Free F1 plan.

**Step 2 — Configure SQL networking:**

- Enabled **Public endpoint** on SQL Server.
- Set **Allow Azure services** to Yes so App Service can connect.
- Added my **client IP** so I could run EF migrations from my local machine.

**Step 3 — Configure App Service settings:**

- Added `ConnectionStrings__DefaultConnection` with the Azure SQL ADO.NET connection string.
- Added JWT settings ( `Jwt__Key` , `Jwt__Issuer` , `Jwt__Audience` ) and `ASPNETCORE_ENVIRONMENT=Production` .
- Used double underscore ( `__` ) naming so .NET configuration binds correctly.

**Step 4 — Prepare code for Production:**

- Updated `Program.cs` so Development uses CORS for local Vite dev server, and Production serves React from `wwwroot` with SPA fallback.
- Added EF Core `Database.Migrate()` on startup to apply schema to Azure SQL.

**Step 5 — Build and deploy:**

- Ran `npm run build` for React, `dotnet publish` for the API.
- Copied `client/dist` into `publish/wwwroot` .
- Zipped publish folder contents and uploaded via App Service **Deployment Center** (OneDeploy).

**Step 6 — Verify:**

- Opened the live URL (secure unique hostname), tested login with seed employee code EMP001.

- Enabled **Application Insights** for monitoring.

**Why this approach:** No GitHub required for first deploy; full control over build artifacts; single App Service URL for both UI and API.

---

## Question 2

**Interviewer:** Why did you use Azure SQL Database instead of your local SQL Express instance for the cloud deployment?

**Context to set:** Shows you understand cloud architecture boundaries and environment separation.

**Answer:**

The Azure-hosted App Service runs in Microsoft's cloud datacenter. It cannot reach my local machine's SQL Express instance ( `DESKTOP-...\SQLEXPRESS` ) because that database sits behind my home network firewall and is not publicly exposed—and exposing it would be a serious security risk.

Therefore I use **two separate databases**:

| Environment | Database | Connection config location |
|---|---|---|
| Local development | SQL Express on my PC | `appsettings.Development.json` |
| Azure production | Azure SQL Database | App Service Environment variables |

When I deployed, EF Core migrations created fresh tables and seed data on Azure SQL. My locally logged activities did not automatically copy over—that is expected unless I explicitly migrate data using BACPAC export/import or scripts.

**Interview principle:** Never point a cloud app at a developer's local database. Use environment-specific connection strings.

---

## Question 3

**Interviewer:** How did you host both the React frontend and .NET API on Azure? Why not use two separate services?

**Context to set:** Demonstrates understanding of static file serving, SPA routing, and architectural trade-offs.

**Answer:**

I used a **single App Service** to host both layers:

```
https://championportal-ai.azurewebsites.net/          → React (index.html, assets)
https://championportal-ai.azurewebsites.net/api/...   → .NET API controllers
```

**Build process:**

1. Build React → output in `client/dist`
2. Publish .NET API → output folder with DLLs
3. Copy `client/dist` into `publish/wwwroot`
4. Zip and deploy to App Service

**Production code in Program.cs:**

```
app.UseDefaultFiles();
app.UseStaticFiles();
app.MapControllers();
app.MapFallbackToFile("index.html");
```

**Why single App Service:**

- One URL—no CORS issues between frontend and API
- Simpler deployment (one zip)
- Lower cost for demos (one Free F1 app)
- React already calls `/api` as relative path—works without code changes

**Alternative (mention in interview):** Azure Static Web Apps for React + separate App Service for API, or Azure Front Door as reverse proxy. I chose simplicity for learning.

**Key concept:** `wwwroot` is ASP.NET Core's default static files folder. The string "wwwroot" does not appear in code—`UseStaticFiles()` serves from it by convention.

---

## Question 4

**Interviewer:** How did you secure connection strings and JWT secrets in Azure? Why not put them in appsettings.json?

**Context to set:** Security is a top interview topic. Show you understand secrets management and the trust boundary between code and configuration.

**Answer:**

**What I did:**

- Stored all production secrets in **App Service Configuration** (Environment variables):
  - `ConnectionStrings__DefaultConnection`
  - `Jwt__Key`, `Jwt__Issuer`, `Jwt__Audience`
  - `ASPNETCORE_ENVIRONMENT=Production`
- Left `appsettings.Development.json` for local dev only (local SQL, dev JWT key).
- Never committed Azure passwords or production keys to git.

**Why double underscore ( `__` ):** In .NET configuration, `ConnectionStrings__DefaultConnection` maps to `ConnectionStrings:DefaultConnection` in JSON hierarchy. Single underscore does not bind—app fails silently or throws on startup.

**Why not appsettings.json in production:**

- Files in deploy zip can be inspected
- Risk of accidental git commit
- No rotation without redeploy
- Violates least-privilege and separation of config from code

**Production improvement (mention proactively):**

- **Azure Key Vault** to store secrets
- **Managed Identity** so App Service reads Key Vault without passwords in config
- Key Vault references in App Service settings: `@Microsoft.KeyVault(...)`

**Interview line:** "Secrets live in Azure configuration, not in source code. Local and cloud use different config sources for the same codebase."

---

## Question 5

**Interviewer:** Explain Development vs Production configuration in your Program.cs. What runs where?

**Context to set:** Shows environment-aware application design—the `ASPNETCORE_ENVIRONMENT` pattern.

**Answer:**

.NET uses `ASPNETCORE_ENVIRONMENT` to branch behavior:

| Environment | Set by | Where |
|---|---|---|
| Development | `launchSettings.json` | Local PC |
| Production | App Service Configuration | Azure |

**On local PC (Development):**

```
if (builder.Environment.IsDevelopment())
{
    builder.Services.AddCors(... localhost:5173 ...);
}
// ...
app.UseSwagger();
app.UseSwaggerUI();
app.UseCors("ViteDev");
```

- React runs on Vite dev server ( `localhost:5173` )
- API runs on Kestrel ( `localhost:5228` )
- CORS connects the two
- Swagger enabled for API testing

**On Azure (Production):**

```
else
{
    app.UseDefaultFiles();
    app.UseStaticFiles();
}
// ...
app.MapFallbackToFile("index.html");
```

- React served from built files in `wwwroot`
- Same origin as API—no CORS needed
- Swagger disabled in production
- SPA fallback so `/dashboard` works on browser refresh

**Also runs in both environments:**

```
db.Database.Migrate();
```

Applies EF Core migrations on startup—creates/updates Azure SQL schema.

**One-line summary:** "Development block = local PC with Vite. Else block = live Azure serving wwwroot."

---

## Question 6

**Interviewer:** You got ERR_NAME_NOT_RESOLVED when opening your App Service URL. What happened and how did you fix it?

**Context to set:** Real troubleshooting story—shows you understand Azure URLs and secure hostnames.

**Answer:**

During App Service creation, **Secure unique default hostname** was enabled. That means my public URL was not the simple:

```
championportal-ai.azurewebsites.net  ❌ (does not resolve)
```

But instead a unique hostname like:

```
championportal-ai-f0d4btecajh3a0es.centralindia-01.azurewebsites.net  ✓
```

**Why:** Azure assigns a non-guessable hostname for security—reduces enumeration attacks on default naming patterns.

**Fix:** Copied the exact **Default domain** from App Service → Properties blade in Azure Portal. Used **Browse** button from Overview to open correct URL.

**Lesson:** Always copy the URL from Portal after creation; do not assume `{appname}.azurewebsites.net` unless you verify it resolves.

**Related:** Custom domains can be added later via DNS CNAME—but not required for demos. The default unique hostname is fully public and shareable.

---

## Question 7

**Interviewer:** What Azure resources did you create and how do they relate to each other?

**Context to set:** Architecture diagram question—show you see the full picture.

**Answer:**

```
Resource Group: rg-championportal (Central India)
│
├── App Service: championportal-AI
│   └── Hosts .NET API + React (wwwroot)
│   └── Linked to → Application Insights: championportal-AI
│
├── App Service Plan: ASP-rgchampionportal-8e98
│   └── Compute plan (Free F1) for championportal-AI
│
```

```
├── SQL Server: championportal-sql-ashish
│   └── Hosts → SQL Database: ChampionPortal (Basic tier)
│
├── Function App: func-demo-http-ashish (Consumption plan)
│   └── Linked to → Application Insights
│   └── Requires → Storage Account: stfuncdemoashish
│
└── Storage Account: stfuncdemoashish (LRS)
    └── Backend for Functions runtime (AzureWebJobsStorage)
```

**Data flow for Champion Portal:**

```
User → App Service → Azure SQL Database
User → App Service wwwroot → React static files
User → App Service /api → JWT auth → EF Core → SQL
```

**Interview tip:** Resource Group is for lifecycle management—delete the group to clean up everything for cost control after practice.

---

## Question 8

**Interviewer:** How did you configure Azure SQL networking and why were those settings important?

**Context to set:** Cloud security and connectivity—firewall rules are frequently asked.

**Answer:**

**Settings applied on SQL Server Networking blade:**

| Setting | Value | Why |
|---|---|---|
| Connectivity method | Public endpoint | Allows App Service and my PC to connect |
| Allow Azure services | Yes | App Service can reach SQL without my home IP |
| Client IP rule | My public IPv4 added | My PC can run EF migrations and SSMS |
| Minimum TLS | 1.2 | Encrypt connections in transit |

**Two different consumers:**

1. **App Service (cloud)** — uses "Allow Azure services" rule
2. **My PC (local)** — uses client IP firewall rule

**If Allow Azure services = No:** Live app gets connection errors (500) even with correct connection string.

**If client IP not added:** `dotnet ef database update` from PC fails with firewall error.

**Security note:** I did not open SQL to `0.0.0.0/0` (entire internet). Only Azure services + my specific IP.

**When IP changes:** Home ISP may assign new IP—add updated rule in SQL Server → Networking.

---

## Question 9

**Interviewer:** Have you used Application Insights? What did you configure and what can you see?

**Context to set:** Monitoring and observability—connects to "how do you debug production issues."

**Answer:**

Yes. I enabled Application Insights on both App Service (Champion Portal) and Function App.

**Setup:**

- App Service → Application Insights → Enable → Create resource `championportal-Insights`
- Linked to Log Analytics workspace (auto-created)
- Restarted app after applying

**What I monitored:**

| Blade | Purpose | What I saw |
|---|---|---|
| Overview | Summary | ~0% failed requests, ~50ms avg response time |
| Live Metrics | Real-time | Request spikes while browsing app |
| Search (Transaction search) | Individual requests | POST /api/auth/login, GET /, dashboard calls |
| Performance | Slow endpoints | Average duration per operation |
| Failures | Errors | 401/500 exceptions with stack traces |

**Debugging workflow:**

1. User reports issue on live URL
2. Check Failures blade for exceptions
3. Drill into specific request in Search
4. Cross-reference Log stream for immediate console output

**Optional demo I performed:** Temporarily broke JWT key → saw auth failures in Failures → restored key → verified recovery.

**Interview line:** "Application Insights gives me request-level tracing from UI action to API failure—not just server uptime."

---

## Question 10

**Interviewer:** Users were logged out on every page refresh after deployment. What was the root cause and fix?

**Context to set:** Full-stack debugging—React state vs persistence, not an Azure issue.

**Answer:**

**Symptom:** Login worked, but browser refresh returned user to login page on Azure live site.

**Root cause:** JWT token and user info stored only in React `useState` in `AuthContext.tsx`. React state is in-memory—it resets on full page reload. This would happen locally too on hard refresh; more noticeable on Azure during testing.

**Not the cause:** Azure, Application Insights, App Service, or CORS.

**Fix applied:**

- Persist auth payload to `sessionStorage` on login

- Restore from `sessionStorage` when `AuthProvider` initializes
- Clear `sessionStorage` on logout

```
const STORAGE_KEY = 'championPortal.auth';
// readStoredAuth() on init
// writeStoredAuth() on login/logout
```

**Also fixed:** Removed duplicate `app.UseCors("ViteDev")` outside Development block in Program.cs—could cause issues in Production where CORS policy is not registered.

**Interview angle:** "Distinguish infrastructure bugs from application state management. Session persistence is a frontend concern—use sessionStorage, localStorage, or httpOnly cookies for production auth."

**Production improvement:** httpOnly secure cookies instead of sessionStorage (XSS protection).

---

## Question 11

**Interviewer:** Why does Azure Functions require a Storage account? What is AzureWebJobsStorage?

**Context to set:** Core Functions concept—you discovered this hands-on when scale-out warning appeared.

**Answer:**

Every Azure Function App **requires** a Storage account connected via the `AzureWebJobsStorage` connection string setting.

**Storage is used by the Functions runtime host for:**

| Purpose | Description |
|---|---|
| Scale coordination | Multiple instances coordinate via storage leases |
| Host locks | Prevent duplicate timer/queue processing |
| Trigger monitoring | Queue, blob, timer triggers watch storage |
| Internal state | Host metadata, sync state, logs |
| Secret storage | When using file-based secret storage type |

**Important distinction:**

- `AzureWebJobsStorage` = **runtime infrastructure** (always required)
- Business data can live in Azure SQL, Cosmos DB, etc.—separate from runtime storage

**What happened in my deployment:** Function App was created without a visible Storage account in the resource group. `AzureWebJobsStorage` was missing from configuration → **"Scale out issues detected"** warning and Portal could not create functions properly.

**Fix:** Manually created Storage account ( `stfuncdemoashish` , LRS), copied connection string from Access keys, added as `AzureWebJobsStorage` connection string, restarted Function App.

**Analogy:** Function App = worker; Storage account = coordinator/brain for scaling and triggers.

---

## Question 12

**Interviewer:** Explain Azure Functions Consumption plan. How is it different from App Service?

**Context to set:** Serverless vs PaaS web hosting—common comparison question.

**Answer:**

**Consumption plan (serverless):**

- Pay only when functions execute (per million executions + GB-seconds)
- **Scales to zero** when idle—no compute charge
- **Cold start** on first request after idle period
- Event-driven automatic scaling
- Best for: webhooks, scheduled jobs, queue processors, lightweight APIs

**App Service (web host):**

- Always-on web server (even on Free tier, though can sleep)
- Full ASP.NET pipeline, middleware, static files, long-running requests
- Better for: full web applications, SPAs with API, WebSockets, complex middleware
- Champion Portal uses App Service because it is a full-stack web app

**Comparison table:**

| Aspect | Consumption Functions | App Service |
|---|---|---|
| Model | Event-driven, serverless | Always-on PaaS host |
| Billing | Per execution | Per plan (Free/Basic/Standard) |
| Idle cost | Zero compute | Plan cost continues |
| Cold start | Yes | Free tier sleep, not true scale-to-zero |
| My use case | HTTP trigger demo | Champion Portal full app |

**Interview line:** "I use App Service for the main web application and Functions for discrete event-driven tasks like webhooks or scheduled notifications."

---

## Question 13

**Interviewer:** Why couldn't you create a function in the Azure Portal for .NET 8 Isolated? What did you do instead?

**Context to set:** Shows you understand runtime limitations and alternative deployment paths.

**Answer:**

When I created a Function App with **.NET 8 Isolated worker model**, the Portal showed only options to create via **Visual Studio, VS Code, or CLI**—no "Develop in portal" button.

**Reason:** .NET Isolated functions compile to a separate worker process. In-portal editing is supported for **JavaScript, Python, and PowerShell** on Consumption plan—not for .NET Isolated.

**What I did:**

1. Deleted the .NET Isolated Function App (empty, not useful without CLI deploy)
2. Created new Function App with **Node.js 20 LTS** on Consumption (Windows) plan
3. Intended to use Portal → Functions → Create → HTTP trigger

**Additional blocker discovered:** Node Function App also lacked `AzureWebJobsStorage` until I manually created Storage account and linked it.

**Alternatives for .NET Functions (mention in interview):**

- Visual Studio → Publish to Azure Function
- VS Code + Azure Functions extension
- Azure Functions Core Tools: `func init`, `func azure functionapp publish`
- Azure Cloud Shell: `az functionapp function create`

**Interview line:** "Runtime choice affects deployment workflow. I understand Portal limitations and can deploy via IDE or Core Tools for .NET Isolated."

---

## Question 14

**Interviewer:** What is the difference between LRS, GRS, and blob storage tiers? When would you use each?

**Context to set:** Storage redundancy and cost optimization—Demo 5 topic integrated here.

**Answer:**

**Redundancy (Storage account level):**

| Tier | Full name | Copies | Use case | Cost |
|---|---|---|---|---|
| LRS | Locally redundant | 1 datacenter, 3 copies | Dev, demo, Functions runtime | Lowest |
| ZRS | Zone redundant | Across availability zones | Higher availability in one region | Medium |
| GRS | Geo-redundant | Secondary region async copy | Disaster recovery | Higher |
| RA-GRS | Read-access GRS | Read from secondary | DR with read failover | Highest |

I chose **LRS** for Function App storage ( `stfuncdemoashish` ) to minimize cost during practice.

**Blob access tiers (within storage account):**

| Tier | Access pattern | Example |
|---|---|---|
| Hot | Frequent | Active user uploads, web assets |
| Cool | Infrequent (30+ days) | Monthly reports |
| Cold | Rare (90+ days) | Old backups |
| Archive | Long-term archive | Compliance retention, slow retrieval |

**Interview decision framework:**

- Functions runtime storage → LRS is sufficient
- Production business blobs with DR requirement → GRS or RA-GRS
- Cost optimization on old files → lifecycle policies to move Hot → Cool → Archive

---

## Question 15

**Interviewer:** How do EF Core migrations work in your Azure deployment? What happens to existing local data?

**Context to set:** Database lifecycle management in cloud deployments.

**Answer:**

**On Azure (fresh database):**

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}
```

On first App Service startup after deploy, EF Core applies all pending migrations to Azure SQL—creates tables, indexes, and seed data (employees EMP001–EMP004, activity types).

**Seed data included:** Activity types and employees from `SeedData.cs` in migrations—not user-created activity logs.

**Local data did not migrate automatically:**

- Activities I logged locally stayed in SQL Express on my PC
- Azure SQL started fresh with schema + seed only
- To migrate local data: BACPAC export/import, SQL scripts, or Azure Data Migration tools

**Alternative to Migrate() on startup:**

- Run `dotnet ef database update` from PC against Azure SQL connection string
- CI/CD pipeline step before deploy
- SQL scripts in release pipeline

**Production caution:** Auto-migrate on startup is convenient for demos but some teams prefer controlled migration in pipeline to avoid race conditions with multiple instances.

**Rollback question (bonus):**

```
dotnet ef database update PreviousMigrationName
```

Or generate SQL script and apply manually for production control.

---

## Question 16

**Interviewer:** Explain JWT authentication configuration in your Azure deployment. How does login work end-to-end?

**Context to set:** Auth flow from React → API → token → protected routes.

**Answer:**

**Champion Portal uses custom JWT authentication (employee code login), not Microsoft Entra ID.**

**Flow:**

1. User enters employee code (e.g. EMP001) on React login page
2. React POSTs to `/api/auth/login`
3. API `LoginCommandHandler` validates employee in database
4. `JwtTokenService` generates signed JWT with claims (employee id, name, reviewer flag)
5. Token returned to React, stored in sessionStorage (after fix)
6. Subsequent API calls include `Authorization: Bearer {token}`
7. ASP.NET JWT middleware validates signature using `Jwt__Key`, issuer, audience

**Azure configuration (not in code):**

```
Jwt__Key       = (32+ character secret)
Jwt__Issuer    = ChampionPortal
Jwt__Audience  = ChampionPortal
```

**Why separate from Entra ID:** Demo app uses simple employee-code auth. Production enterprise apps would use **Microsoft Entra ID (Azure AD)** with OAuth2/OIDC and Easy Auth on App Service.

**Security practices applied:**

- Production JWT key different from dev key in appsettings.Development.json
- Key stored in App Service Configuration, not source code
- HTTPS enforced on Azure URL

**If JWT key too short or wrong:** API returns 500 or 401; visible in Application Insights Failures.

---

## Question 17

**Interviewer:** What troubleshooting steps did you take during deployment? How would you debug a 500 error on the live site?

**Context to set:** Operational maturity—structured debugging, not guessing.

**Answer:**

**Issues encountered and resolution:**

| Issue | Diagnosis | Fix |
|---|---|---|
| ERR_NAME_NOT_RESOLVED | Wrong assumed URL | Used Default domain from Portal |
| Logout on refresh | Token in React state only | sessionStorage persistence |
| App settings not working | Single _ instead of __ | Renamed to double underscore |
| Functions scale warning | Missing AzureWebJobsStorage | Created Storage account, linked connection |
| No Create function in Portal | .NET Isolated + missing storage | Node.js app + storage fix |

**500 error debugging workflow:**

1. **Reproduce** — note exact URL and action
2. **Log stream** — App Service → real-time console errors
3. **Application Insights** → Failures → exception message and stack trace
4. **Check configuration** — connection string, JWT settings, environment name
5. **Check SQL** — firewall rules, credentials, migrations applied
6. **Check deploy artifact** — wwwroot present, correct zip structure
7. **Fix and redeploy** — rebuild zip if code change needed

**Common 500 causes in my stack:**

- Invalid/missing connection string
- SQL firewall blocking Azure
- JWT key missing or invalid
- EF migration failure on startup
- CORS misconfiguration (less likely with single App Service)

**Interview line:** "I start with Log stream for immediate signal, then Application Insights for request-level exception detail."

---

## Question 18

**Interviewer:** Why did you choose Azure SQL Basic tier? What other options exist and when would you upgrade?

**Context to set:** Cost vs performance trade-offs in Azure SQL.

**Answer:**

**What I chose:**

- **Basic tier** — 5 DTUs, 2 GB storage, ~$5.60/month
- **LRS** backup redundancy
- **Development** workload environment label
- **No elastic pool** — single database sufficient

**Why Basic for practice:**

- Lowest cost relational tier in Azure SQL
- Sufficient for demo traffic and EF Core app
- Predictable monthly cost vs Serverless variable billing
- Full SQL Server compatibility for EF Core

**Alternatives considered:**

| Tier | When to use |
|---|---|
| Basic | Dev, demo, low traffic |
| Standard (S0–S12) | Production moderate load |
| Premium | IO-intensive, low latency |
| Serverless (GP) | Variable/intermittent workload, auto-pause |
| Hyperscale | Very large databases, fast growth |

**When I would upgrade:**

- Slow queries under load → Standard or higher DTUs/vCores
- Need geo-replication → add secondary replica
- Production SLA → Standard tier minimum with backup retention policies
- High concurrency → connection pooling tuning + higher tier

**Interview note:** I initially landed on Serverless GP (~$5.45+ variable) before switching to Basic for predictable demo cost.

---

## Question 19

**Interviewer:** What is the difference between App Service, App Service Plan, and Resource Group? Why do you need all three?

**Context to set:** Fundamental Azure resource hierarchy—often confused by beginners.

**Answer:**

**Resource Group:**

- Logical **container** for related resources
- Used for deployment, access control (RBAC), billing tags, and delete-as-a-unit
- My group: `rg-championportal`
- Does not execute code or host apps by itself

**App Service Plan:**

- Defines **compute resources** (CPU, RAM, region, pricing tier)
- Examples: Free F1, Basic B1, Standard S1
- My plans: `ASP-rgchampionportal-8e98` (Free F1 for web app)
- Multiple App Services can share one plan (same region and tier)

**App Service (Web App):**

- The actual **running application host**
- My app: `championportal-AI`
- Runs .NET 9, serves HTTP, holds configuration
- Must be linked to exactly one App Service Plan

**Analogy:**

```
Resource Group  = folder on your desk
App Service Plan = electricity plan / server size contract
App Service     = the running website plugged into that plan
```

**Function App note:** Function App on Consumption plan gets its own hidden/y1 plan ( `ASP-rgchampionportal-a1aa` )—still an App Service Plan under the hood, but billed per execution.

**Why separate plan from app:** Scale or change pricing tier for multiple apps together; swap plans for production scaling without recreating the app.

---

## Question 20

**Interviewer:** If you had one more week to improve this Azure deployment for production, what would you do?

**Context to set:** Shows forward thinking, production readiness, and awareness of gaps—strong closing answer.

**Answer:**

**Week 1 production hardening roadmap:**

**Security:**

1. Move secrets to **Azure Key Vault** with **Managed Identity**
2. Replace employee-code JWT with **Microsoft Entra ID** (OAuth2/OIDC)
3. Enable **HTTPS only**, review SQL firewall (remove client IP if not needed)
4. Rotate SQL password and JWT keys after any exposure

**Reliability:** 5. Upgrade App Service from **Free F1** to **Basic/Standard** (always-on, no sleep) 6. Add **deployment slots** (staging → production swap) 7. Configure **auto-heal** and health check endpoints 8. Azure SQL **Standard tier** with automated backups and retention policy

**CI/CD:** 9. Push code to **GitHub** → **Azure DevOps** or GitHub Actions pipeline 10. Automated: build React → publish API → test → deploy to staging slot → swap

**Monitoring:** 11. Full **Application Insights** alerts (failure rate, response time, SQL dependency failures) 12. **Azure Monitor** action groups for email/Teams notifications 13. Dashboard for ops team

**Architecture:** 14. **Azure Service Bus** for async activity processing if load grows 15. **Azure Functions** for scheduled reports or webhook integrations 16. Consider **Front Door** or **CDN** for static assets if global users

**Data:** 17. BACPAC backup schedule for Azure SQL 18. Document migration rollback procedure

**Interview closing line:** "My current deployment proves end-to-end capability. Production readiness means Key Vault, Entra ID, CI/CD, staged deployments, and proactive monitoring—not just a working zip deploy."

---

## Quick Reference — Terms Used in This Guide

| Term | Definition |
|---|---|
| App Service | PaaS web host for .NET, Node, etc. |
| App Service Plan | Compute SKU (F1, B1, S1) |
| Resource Group | Logical Azure container |
| Azure SQL Database | Managed SQL Server in cloud |
| wwwroot | ASP.NET default static files folder |
| AzureWebJobsStorage | Required Functions → Storage connection |
| Consumption plan | Serverless Functions billing model |
| Application Insights | APM / telemetry service |
| Double underscore __ | .NET env var → nested config mapping |
| MapFallbackToFile | SPA route refresh support |
| LRS / GRS | Storage redundancy levels |
| Secure unique hostname | Non-default azurewebsites.net URL |

| | |
|---|---|
| OneDeploy | Azure ZIP deployment mechanism |
| Cold start | First-request delay on Consumption |
| Scale to zero | No compute instances when idle |

---

# Appendix — Supplement: 3 Core Azure Questions + Follow-Ups

See also standalone file: `Azure-Interview-Supplement-3-Questions.pdf`

---

## Question A — Azure Functions & Its Triggers

**Interviewer:** Explain Azure Functions and its triggers. When would you use Functions instead of App Service?

**Context to set:** Mention Consumption plan, HTTP trigger, AzureWebJobsStorage, .NET Isolated vs Node.js portal limits.

**Answer:** Azure Functions is serverless, event-driven compute. A **Function App** hosts one or more **functions**, each with a **trigger** (HTTP, Timer, Queue, Blob, Service Bus, Event Grid, Cosmos DB, Event Hub). On **Consumption plan**, pay per execution, scale to zero, possible cold start. **AzureWebJobsStorage** is mandatory for runtime coordination. I created `func-demo-http-ashish` (Node.js, Consumption). .NET Isolated requires VS Code/CLI, not Portal create. Use **App Service** for full web apps (Champion Portal); **Functions** for webhooks, schedules, queue handlers.

**Follow-ups:** Cold start mitigation | AzureWebJobsStorage purpose | HTTP auth levels (Anonymous/Function/Admin) | Portal limits for .NET | Functions + Service Bus example for async activity processing

---

## Question B — Azure Service Bus / RabbitMQ

**Interviewer:** Compare Azure Service Bus and RabbitMQ. What happens when a Service Bus message fails?

**Context to set:** Champion Portal uses sync API → SQL today; messaging is the scale-up pattern.

**Answer:** Both decouple producers/consumers via messages. **Service Bus** (Azure PaaS): queues (point-to-point), topics/subscriptions (pub/sub), **dead-letter queue (DLQ)** for failed messages after retries. **RabbitMQ**: open-source, self-hosted, exchanges/queues/bindings. On failure: retry → DLQ → investigate → fix idempotent handler → reprocess. Use Service Bus in Azure-native apps; RabbitMQ for multi-cloud or existing RabbitMQ teams.

**Follow-ups:** Queue vs Topic | Poison message | Idempotency | Service Bus vs Storage Queue | When messaging is overkill

---

## Question C — Have You Used Azure Application Insights?

**Interviewer:** Have you used Application Insights? What did you monitor?

**Context to set:** Strong yes — enabled on championportal-AI with hands-on verification.

**Answer:** Yes. Enabled **championportal-Insights** on App Service. Used **Overview** (failed requests, response time), **Live Metrics** (real-time), **Search** (individual `/api/auth/login` traces), **Performance** (slow ops), **Failures**

(exceptions). Debug flow: Failures → Search drill-down → Log stream → fix → verify. Production add: alerts, availability tests, KQL dashboards. Connection via `APPLICATIONINSIGHTS_CONNECTION_STRING`.

**Follow-ups:** Insights vs Log stream | Dependencies (SQL calls) | Find slow endpoint | Alert rules | Functions + Insights

---

## Mock Drill — Rapid Fire

| Question | Answer cue |
|---|---|
| 4 Function triggers? | HTTP, Timer, Queue, Blob |
| Scales to zero? | Consumption plan |
| Functions storage setting? | AzureWebJobsStorage |
| Failed message destination? | Dead-letter queue |
| Real-time monitoring blade? | Live Metrics |
| Individual request traces? | Search |

---

*Document generated from hands-on Champion Portal Azure deployment practice.*